

Oroex and FortVault - High-Level Platform Architecture

Purpose

Oroex is planned as a light crypto exchange / swap product. Users can deposit crypto, swap one asset to another, and withdraw funds. Oroex does not operate its own order book. Instead, Oroex uses FortVault as a custody and processing provider, and FortVault integrates with external exchanges for liquidity and mirror trade execution.

FortVault is the secure infrastructure layer behind Oroex. It provides workspace management, vaults, customer address generation, blockchain processing, MPC signing, exchange integration, custody operations, and operational audit trails.

Main Principle

Oroex owns product, users, UX, swaps, and user ledger.

FortVault owns custody, processing, vaults, exchange credentials, blockchain execution, and security.

Oroex should not directly hold blockchain private keys, MPC keys, or exchange API secrets.

High-Level Components

Oroex Applications

Oroex will have:

- iOS mobile application.
- Android mobile application.
- Web application.
- Web-based admin panel.

Oroex Backend

Oroex backend is the product API layer.

It owns:

- User-facing product logic.
- Swap flow.
- Quote display and quote acceptance.
- Product fees and spreads.
- User ledger.
- Available and locked balances.
- User withdrawal requests.
- Integration with FortVault APIs.
- Handling signed webhooks/events from FortVault.

Oroex User Management

Oroex user management owns:

- User registration.

- Login/authentication.
- User profile.
- KYC/AML state if required.
- User limits and restrictions.
- Mapping between Oroex users and FortVault customers.

For MVP, user management can be part of Oroex backend. Later it can be separated into a dedicated service.

FortVault

FortVault acts as Oroex custody and processing provider.

It owns:

- Oroex workspace/partner configuration.
- Custody mode configuration.
- Vault creation and management.
- Customer records created through API.
- Customer deposit address generation.
- Blockchain listener events.
- Deposit detection and confirmation.
- Auto-sweep execution if enabled.
- On-chain withdrawal execution.
- MPC signing.
- Exchange account management.
- Exchange API credential protection.
- Exchange trade execution.
- Exchange balance synchronization.
- Operational history and audit logs.

FortVault Workspace Model

Oroex will be configured as a dedicated FortVault workspace / partner.

```
FortVault Workspace: Oroex
  Vaults
  Customers
  Assets
  Chains
  Exchange Accounts
  Custody Mode Settings
  Auto-Sweep Settings
  Policies
```

Oroex admins can use the existing FortVault dashboard to configure operational infrastructure.

Custody Structures

FortVault should support two custody structures for Oroex, similar to Fireblocks-style custody models.

Segregated Mode

In Segregated mode, each Oroex user is represented with a separated FortVault

customer/address structure.

```
Oroex User A -> FortVault Customer A -> User-specific deposit addresses
Oroex User B -> FortVault Customer B -> User-specific deposit addresses
```

This mode gives clearer separation of customer deposit addresses and easier traceability.

Omnibus Mode

In Omnibus mode, funds are physically pooled into shared FortVault vaults, while Ororex tracks user ownership in its own ledger.

```
User deposits -> customer address -> auto-sweep -> Ororex main vault
```

The physical funds are pooled, but the Ororex user ledger still records exact customer ownership.

Auto-Sweep

Auto-sweep can be enabled per FortVault workspace.

Configuration examples:

- autoSweepEnabled.
- sweepTargetVaultId.
- sweepMinAmountByAsset.
- sweepConfirmationsByChain.
- sweepMode.

Auto-sweep changes physical custody location. It does not change user ownership in Ororex ledger.

Vault Management

Ororex admins create vaults using the FortVault dashboard.

Example vaults:

- Main vault.
- Treasury vault.
- Hot vault.
- Warm vault.
- Cold vault.
- Liquidity vault.
- Exchange funding vault.

These vaults are operational custody containers. They are not the same as Ororex user balances.

Exchange Account Management

Ororex admins create and manage integrated exchange accounts through the FortVault dashboard.

FortVault stores and protects exchange credentials.

Exchange account configuration should include:

- Exchange provider, for example MEXC, Binance, Gate, Kraken.

- Account name.
- API key.
- Encrypted secret or key reference.
- Permissions: read balances, trade, withdraw.
- Status: active, disabled, archived.
- Optional IP whitelist notes.
- Workspace/partner ownership.

Oroex should never store exchange API secrets directly.

User Ledger Ownership

Oroex should own the product user ledger.

The Oroex ledger records:

- User available balances.
- User locked balances.
- Deposit credits.
- Withdrawal debits.
- Swap debits.
- Swap credits.
- Fees.
- Reversals.
- Adjustments.

FortVault records custody and processing facts:

- Blockchain transaction received.
- Address generated.
- Funds swept.
- Withdrawal submitted.
- Withdrawal confirmed.
- Exchange order created.
- Exchange order filled.

The Oroex ledger answers:

Which user owns how much?

FortVault answers:

Where are the funds physically and what processing happened?

FortVault API Integration Flow

1. Workspace Setup

Oroex gets a dedicated FortVault workspace.

Workspace settings define:

- Custody mode: Segregated or Omnibus.
- Auto-sweep enabled or disabled.
- Supported assets and chains.
- Policies and approval rules.
- Exchange integrations.

2. Vault Setup

Oroex admins create operational vaults in FortVault dashboard.

Example:

```
Main
Treasury
Hot
Cold
Liquidity
Exchange Funding
```

3. Exchange Setup

Oroex admins create exchange accounts in FortVault dashboard.

FortVault stores credentials and executes exchange operations through processing workers.

4. Customer Creation

When a user registers in Oroex, Oroex calls FortVault to create a matching FortVault customer.

Example API intent:

```
POST /customers
externalUserId = <oroex-user-id>
```

FortVault stores the external user reference so later events can be mapped back to Oroex.

5. Deposit Address Generation

When a user requests a deposit address in Oroex, Oroex calls FortVault to generate or retrieve an address.

Example API intent:

```
POST /customers/{customerId}/addresses
asset = USDT
chain = Tron
```

FortVault creates the address through its normal customer address generation flow.

6. Deposit Notification

When the user deposits funds, FortVault detects and confirms the transaction.

FortVault then sends a signed webhook/event to Oroex.

Example event intent:

```
deposit.confirmed
customerExternalUserId = <oroex-user-id>
asset = USDT
chain = Tron
amount = 1000
txHash = ...
```

```
confirmations = ...
```

Oroex validates the webhook signature and credits its user ledger.

7. Auto-Sweep Execution

If auto-sweep is enabled, FortVault processing can sweep funds from customer deposit addresses to the configured main vault.

Example:

```
Customer deposit address -> Oroex Main Vault
```

Oroex user ledger remains unchanged. Only physical custody location changes.

8. Swap Request

When a user requests a swap, Oroex performs product-level checks.

Oroex checks:

- User available balance.
- User restrictions.
- Supported pair.
- Product limits.
- Quote expiration.
- Fees and spread.

Oroex then creates a swap order and locks user balance in Oroex ledger.

9. Mirror Exchange Execution

Oroex requests FortVault to execute the required mirror exchange operation.

Example API intent:

```
POST /exchange-orders
exchangeAccountId = MEXC Main
side = BUY
pair = BTCUSDT
quoteAmount = 1000 USDT
maxSlippage = 0.3%
externalSwapId = <oroex-swap-id>
idempotencyKey = ...
```

FortVault validates that the exchange account, pair, amount, and policy are allowed.

FortVault processing executes the order through the exchange API, signs requests using HMAC or Ed25519 depending on exchange, and reports status back to Oroex.

10. Swap Completion

After exchange execution completes, FortVault sends a signed event to Oroex.

Example event intent:

```
exchange.order.filled
externalSwapId = <oroex-swap-id>
fromAsset = USDT
toAsset = BTC
executedAmount = ...
```

```
averagePrice = ...  
fee = ...
```

Oroex finalizes the user ledger:

- Debits source asset.
- Credits destination asset.
- Applies fees.
- Releases locked balance.

11. Withdrawal Request

When a user requests withdrawal, Oroex validates the user ledger balance and creates a withdrawal intent.

Oroex then calls FortVault.

Example API intent:

```
POST /withdrawals  
customerId = ...  
asset = USDT  
chain = Tron  
amount = 100  
toAddress = T...  
idempotencyKey = ...
```

FortVault handles custody checks, policies, approvals if required, MPC signing, broadcast, and status tracking.

12. Withdrawal Status Notification

FortVault sends signed webhook events back to Oroex.

Example events:

```
withdrawal.pending_approval  
withdrawal.broadcasted  
withdrawal.confirmed  
withdrawal.failed
```

Oroex updates user ledger and status based on the result.

API Security Requirements

All Oroex -> FortVault API calls should use:

- Workspace-scoped API credentials.
- Strong authentication.
- Idempotency keys for create/execute requests.
- Request timestamps.
- Request signatures where needed.
- Partner/workspace scoping.

All FortVault -> Oroex webhooks should use:

- Signed webhook payloads.
- Timestamp validation.
- Replay protection.

- Event IDs for idempotency.
- Retry-safe delivery.

Processing Boundary

FortVault processing executes external and asynchronous operations.

Processing is responsible for:

- Auto-sweep jobs.
- Blockchain transaction execution.
- MPC signing requests.
- Exchange API calls.
- Exchange order status polling.
- Exchange balance synchronization.
- Retry and idempotency handling.
- Execution result events.

Processing should not own Ororex user ledger. It should emit verified facts that Ororex ledger can consume.

Accounting Boundary

Ororex ledger owns product balances.

FortVault custody records own custody facts.

Example:

Deposit confirmed:

FortVault event: address A received 1000 USDT

Ororex ledger: user I +1000 USDT

Auto-sweep completed:

FortVault event: funds moved to Main Vault

Ororex ledger: no ownership change

Swap completed:

FortVault event: mirror exchange order filled

Ororex ledger: user I -1000 USDT, +BTC

Exchange Connectivity

For MVP, exchange integration should use HTTPS REST APIs.

REST is used for:

- Account balances.
- Market orders.
- Order status.
- Deposit addresses.
- Withdrawals.
- Withdrawal history.
- Asset and network metadata.

WebSocket can be added later for:

- Live prices.
- Faster balance updates.
- Order execution updates.

- Market data streaming.

Exchange Signing

Different exchanges use different authentication models.

Examples:

- MEXC: HMAC-SHA256.
- Gate: HMAC-SHA512.
- Kraken: HMAC-SHA512-based signing.
- Binance: HMAC or self-generated Ed25519/RSA API keys.

For MVP:

- HMAC exchanges can be supported using protected exchange secrets.
- Binance Ed25519 self-generated keys can later use FortVault MPC EdDSA signing.

Provider Independence

Oroex should treat FortVault as the first processing provider, not as a hardcoded product dependency.

Future possible providers:

- FortVault.
- Fireblocks.
- AlphaPo.
- Other custody or processing providers.

Oroex should depend on a provider abstraction:

```
createCustomer
createDepositAddress
requestWithdrawal
requestExchangeExecution
getOperationStatus
receiveProviderEvents
```

FortVault is the first implementation of that provider interface.

Summary

Oroex:

Product, users, swaps, user ledger, UX, admin experience.

FortVault:

Workspace, custody, vaults, addresses, MPC, processing, exchange credentials, execution, audit

Integration:

Oroex calls FortVault APIs.

FortVault sends signed events back to Oroex.

This architecture keeps Oroex flexible as a product while using FortVault as the secure custody and processing infrastructure.